CTF Challenge Write-Up

Reverse Engineering Challenges: spec ops, coffee grinder, chicken scratch

Problem ID	Captured Flag	Steps (Summary)
P1 – spec ops	`fsuCTF{b14ck_bel7_in_b1ackb0x}`	Provide input.txt, regenerate srand(42) key, reverse XOR and byte-reversal to recover correct input
P2 – coffee grinder	`fsuCTF{1m_g0nn4_n33d_n3w_k1dn3y5}`	Decompile JAR, identify SHA-256('adenosine') key and AES/ECB/PKCS5 encryption, decrypt encrypted_secret.bin
P3 – chicken scratch	`fsuCTF{7hi5_Is_Ju5t_Chick3n_Scr47ch}`	Unpack PyInstaller executable, extract substitution cipher dictionary, decode stored symbol sequence automatically

P1 – spec ops

When the binary runs, it immediately exits unless a file called ./input.txt exists. Once provided, the program reads the file contents and processes the data through a multi-step transformation before comparing against stored ciphertext.

Transformation Analysis

The binary applies the following operations to the input:

```
Step 1: Generate pseudo-random key
  srand(42)
  key[i] = rand() & 0xFF  (32 bytes)

Step 2: XOR and reverse
  out[len-1-i] = input[i] ^ key[i & 0x1F]

Step 3: Compare out[] against stored ciphertext
```

Solution

1. Recreate the 32-byte key by seeding srand(42) and calling rand()
2. XOR the stored ciphertext bytes with the key
3. Reverse the byte order of the result
4. The reversed output is the correct input and contains the flag

Flag: **fsuCTF{b14ck_bel7_in_b1ackb0x}**

P2 – coffee grinder

This challenge was a Java JAR file. Decompiling the archive revealed the encryption scheme used to protect the embedded flag file.

JAR Contents

```
Main.class
CryptUtils.class
encrypted_secret.bin
```

Encryption Scheme

Decompiling the classes revealed the full encryption process:

```
Key derivation:  SHA-256("adenosine")
Algorithm:       AES/ECB/PKCS5Padding
Ciphertext:      encrypted_secret.bin
```

Because both the key derivation method and the encrypted data are known, decryption is straightforward:

Solution

1. Decompile Main.class and CryptUtils.class using javap or a Java decompiler
2. Identify the key derivation: SHA-256 of the string 'adenosine'
3. Identify the algorithm: AES/ECB/PKCS5Padding
4. Compute SHA-256('adenosine') to get the 32-byte AES key
5. Decrypt encrypted_secret.bin using AES/ECB with that key
6. The decrypted output contains the flag

Flag: **`fsuCTF{1m_g0nn4_n33d_n3w_k1dn3y5}`**

P3 – chicken scratch

This binary was a PyInstaller-packed Python executable. The flag was hidden behind a substitution cipher implemented as a Python dictionary mapping strange symbol strings to characters.

Unpacking

The PyInstaller executable was unpacked to extract the embedded Python bytecode and source:

```
$ pyinstxtractor chicken_scratch
$ pycdc chicken_scratch.pyc  # decompile bytecode to source
```

Analysis

The decompiled source revealed:

1. A dictionary mapping symbol strings to plaintext characters (the substitution cipher)
2. A stored sequence of symbols representing the encoded flag
3. The program normally prompts the user to decode each symbol interactively

Solution

4. Extract the substitution dictionary from the decompiled source
5. Extract the stored symbol sequence
6. Apply the dictionary to decode the symbol sequence automatically (no interactive input needed)
7. The decoded message is the flag

Flag: **`fsuCTF{7hi5_Is_Ju5t_Chick3n_Scr47ch}`**